

Week 6: Concurrent Memory Management Simulation

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

Aug 16th, 2026

Overview

This activity prototypes a lightweight memory management model for a custom runtime targeting an embedded system that cannot rely on built-in garbage collection. The program, `mark_sweep_simulation.py`, launches several application threads that concurrently create, link, and release objects inside a shared memory pool of fixed size, while a dedicated collector thread reclaims unreachable memory using a stop-the-world mark-and-sweep cycle. Threading primitives coordinate the two populations of threads so that the heap is never observed in an inconsistent state. The sections below describe the design, present the annotated program output, and evaluate what the simulation demonstrates about memory management under concurrency.

Simulation Structure

Responsibility is divided among three types, each owning one concern:

Component	Responsibility
HeapObject	One allocation: its identifier, the thread that created it, its size in bytes, the set of objects it references, and the mark bit used during collection.
MemoryPool	The heap itself: a fixed 256-byte budget, the table of live objects, a per-thread root set modeling each thread's stack frame, and the allocation and collection routines.
MutatorGate	Phase control: permits many application threads to run at once, but lets the collector freeze all of them for the duration of a collection.

An object moves through six stages, and the trace prints an event at each transition:

1. Request — a thread asks the pool for a given number of bytes.
2. Reservation — the pool verifies the budget and commits the bytes atomically (ALLOC).
3. Construction — the object is created and rooted in the requesting thread's frame (CREATE).
4. Mutation — the object may be linked beneath a parent, making it reachable indirectly (LINK).
5. Release — the thread drops its root reference (DROP). Nothing is freed here; the object merely becomes a candidate.
6. Reclamation — a later collection finds it unreachable and returns its bytes to the pool (RECLAIM).

Two design decisions shape everything else. First, the pool is bounded. A fixed capacity means exhaustion is reachable within a run of only a few dozen allocations, so the trace shows the allocator refusing a request and then recovering, the behavior that matters most on a memory-constrained target. Second, each object carries a set of references to other objects, which makes liveness a question of *graph reachability* rather than a simple flag. An object that no thread holds can still be live because a live parent points at it, and two objects can reference each other while both are garbage. Getting that distinction right is the entire job of the collector.

Multithreaded Python Program

Five threads run concurrently:

Thread	Count	Role
MainThread	1	Launches the workers, joins them, then runs a final shutdown sweep with an empty root set.
Mutator-1..3	3	Application threads: allocate objects, link them together, release them.
Collector	1 (daemon)	Periodic stop-the-world mark-and-sweep every 40 ms.

Each mutator performs five steps. Every step allocates an object of 16 to 40 bytes and then assigns it one of three fates, chosen so that all the interesting reachability cases actually occur in a short run:

- Retained — the object stays rooted in the thread's frame.
- Adopted — the object is linked beneath one the thread already holds and its direct root is dropped, so it survives only through its parent.
- Abandoned in a cycle — the object is paired with the previous one into a mutual reference and both roots are dropped.

Threads sleep briefly between operations to simulate real work and to produce genuinely interleaved execution rather than three sequential bursts.

A note on the GIL: Contrary to popular belief, Python's Global Interpreter Lock (GIL) does not automatically prevent race conditions. The GIL only ensures that individual bytecode instructions execute atomically. Multi-step operations—like `self._used += size` or the

check-and-set logic in `allocate()`—require multiple bytecodes and can still be interrupted.

Therefore, the explicit locks used in this code are actively protecting data, not just acting as boilerplate.

Concurrent Access

Three primitives operate at three different granularities:

Primitive	Protects	Hazard prevented
<code>threading.Lock</code> (<code>EventLog</code>)	Standard output	Interleaved, unreadable log lines. A single <code>print()</code> performs several writes and is not atomic.
<code>threading.RLock</code> (<code>MemoryPool</code>)	Object table, byte counter, root sets, statistics	Lost updates and pool overflow. Reentrant because <code>collect()</code> calls helpers that reacquire the same lock.
<code>threading.Condition</code> (<code>MutatorGate</code>)	Execution phase	The collector tracing a heap that is being mutated underneath it.

The central race lives in `allocate()`. The sequence “test whether the request fits, then commit the bytes” is a check-then-act. If two threads each ran the test before either committed, both would pass and the pool would exceed its declared budget. Enclosing the test and the commit in one critical section removes the window. The same reasoning applies to every statistic: `total_allocated += 1` is a read, an add, and a write, and losing one increment silently corrupts the final accounting.

Writer preference

Once a collection is pending, arriving mutators wait at the gate rather than slipping in ahead of the collector. Without this, a steady stream of allocations could starve the collector indefinitely and the pool would fill even though reclaimable garbage was present.

Lock ordering and self-deadlock

Threads always acquire the gate before the pool lock and never the reverse; a consistent global ordering is what prevents the classic two-lock deadlock. A subtler hazard is that a thread already counted as an active mutator cannot request a collection, because stopping the world waits for the active count to fall to zero and the requester would be waiting on itself. Each worker therefore leaves its critical section before asking for a collection. The trace shows the resulting handshake at 44.68 ms: the collector announces that it is waiting for one active mutator, and roughly 2.8 ms later Mutator-3 reaches the gate and parks until the collection completes.

Mark-and-Sweep Memory Cleanup

Mark. Every mark bit is cleared. The union of all per-thread root sets becomes the initial worklist. The collector then traverses breadth-first, following each object's references and marking everything it reaches. Whatever remains unmarked when the worklist empties is unreachable from any thread.

Sweep. Each unmarked object is removed from the object table and its bytes are returned to the pool's free budget.

Why tracing instead of reference counting? This run includes two deliberate reference cycles to highlight the difference. At 43.78 ms, Mutator-2 links obj#5 and obj#7, then drops their roots. Because they still reference each other, a reference-counting collector would see a non-zero count and permanently leak those 56 bytes. A tracing collector, however, checks reachability from the root set. Finding them unreachable, it successfully sweeps both in cycle 1. A similar cycle (obj#4 and obj#8) is reclaimed in cycle 2. This cleanly illustrates why production runtimes rely on tracing.

Transitive liveness and transitive death. At 32.10 ms obj#6 is linked beneath obj#1 and its direct root is dropped. Cycle 1 nevertheless reports six of eight objects reachable, because obj#1 still points at obj#6 even though no thread holds obj#6 directly. At 71.33 ms Mutator-1 drops obj#1; obj#6 loses its last path from any root, and cycle 2 sweeps both. Liveness propagates forward along references, and so does death.

Why the world must stop. If a mutator could root a new object after the collector had already traversed that region of the graph, the object would finish the mark phase unmarked and be swept while still in use, a use-after-free introduced by the collector itself. Freezing the mutators for the duration of the trace is the simplest correct answer, and it is what this simulation implements.

Runtime Events

Every significant transition is printed through a single lock-protected sink, tagged by category so the interleaved trace remains readable:

Tag	Meaning
THREAD	Thread lifecycle and coordination (start, finish, safepoint pause)
ALLOC	The pool grants bytes for a new object
CREATE	The object is constructed and rooted in a thread's frame
LINK	One object stores a reference to another
DROP	A thread releases a root reference; the object may now be garbage
OOM	Allocation refused because the pool is exhausted
GC-STOP	Stop-the-world requested, mutators paused, world resumed
GC-MARK	Mark phase tracing reachability from the root set
RECLAIM	Sweep phase reclaiming an unreachable object's memory
GC-DONE	Collection finished; pool utilization reported

Each line carries a millisecond timestamp and the name of the thread that produced it, so the interleaving of the four active threads is directly visible.

```

python3 mark_sweep_simulation.py  -- 1 of 3

=====
CS 660 Module Six Activity -- Concurrent Mark-and-Sweep Memory Simulation
=====

Pool capacity      : 256 B
Mutator threads    : 3 (5 operations each)
Collector          : 1 background thread, stop-the-world, 40 ms interval

Event legend
THREAD  thread lifecycle and coordination
ALLOC   pool grants bytes for a new object
CREATE  object constructed and rooted in a thread's frame
LINK    one object stores a reference to another
DROP    thread releases a root reference (object may be garbage)
OOM      allocation refused -- pool exhausted
GC-STOP  stop-the-world requested / mutators paused
GC-MARK  mark phase: tracing reachability from the root set
RECLAIM  sweep phase: unreachable object's memory reclaimed
GC-DONE  collection finished, pool utilization reported

[ 0.09 ms] Collector  THREAD  collector online -- periodic sweep every 40 ms
[ 0.12 ms] MainThread THREAD  main launching 3 mutator thread(s)
[ 0.17 ms] Mutator-1  THREAD  started -- entering allocate/release loop
[ 0.18 ms] Mutator-1  ALLOC   reserved 16B for obj#1 -> pool at 16/256B
[ 0.19 ms] Mutator-1  CREATE  obj#1 constructed and rooted in Mutator-1's frame
[ 0.25 ms] Mutator-2  THREAD  started -- entering allocate/release loop
[ 0.26 ms] Mutator-2  ALLOC   reserved 24B for obj#2 -> pool at 40/256B
[ 0.27 ms] Mutator-2  CREATE  obj#2 constructed and rooted in Mutator-2's frame
[ 0.31 ms] Mutator-3  THREAD  started -- entering allocate/release loop
[ 0.32 ms] Mutator-3  ALLOC   reserved 16B for obj#3 -> pool at 56/256B
[ 0.32 ms] Mutator-3  CREATE  obj#3 constructed and rooted in Mutator-3's frame
[ 14.85 ms] Mutator-3  ALLOC   reserved 32B for obj#4 -> pool at 88/256B
[ 14.86 ms] Mutator-3  CREATE  obj#4 constructed and rooted in Mutator-3's frame
[ 19.26 ms] Mutator-2  ALLOC   reserved 16B for obj#5 -> pool at 104/256B
[ 19.28 ms] Mutator-2  CREATE  obj#5 constructed and rooted in Mutator-2's frame
[ 25.85 ms] Mutator-1  ALLOC   reserved 32B for obj#6 -> pool at 136/256B
[ 25.87 ms] Mutator-1  CREATE  obj#6 constructed and rooted in Mutator-1's frame
[ 32.10 ms] Mutator-1  LINK    obj#1 now references obj#6
[ 32.12 ms] Mutator-1  DROP    Mutator-1 released obj#6 -- candidate for sweeping
[ 36.39 ms] Mutator-2  ALLOC   reserved 40B for obj#7 -> pool at 176/256B
[ 36.41 ms] Mutator-2  CREATE  obj#7 constructed and rooted in Mutator-2's frame
[ 40.08 ms] Mutator-3  ALLOC   reserved 16B for obj#8 -> pool at 192/256B
[ 40.09 ms] Mutator-3  CREATE  obj#8 constructed and rooted in Mutator-3's frame
[ 43.76 ms] Mutator-2  LINK    obj#5 now references obj#7
[ 43.77 ms] Mutator-2  LINK    obj#7 now references obj#5
[ 43.78 ms] Mutator-2  THREAD  built reference cycle obj#5 <-> obj#7, abandoning both
[ 43.79 ms] Mutator-2  DROP    Mutator-2 released obj#5 -- candidate for sweeping
[ 43.79 ms] Mutator-2  DROP    Mutator-2 released obj#7 -- candidate for sweeping

```

Figure 1. Startup and concurrent allocation. Three mutators interleave allocations into the shared pool. At 32.10 ms obj#6 is linked beneath obj#1 and its root dropped; at 43.76 ms Mutator-2 builds the reference cycle obj#5 $\leftrightarrow$ obj#7 and abandons both.

```

python3 mark_sweep_simulation.py  - 2 of 3

[ 44.65 ms] Collector    GC-STOP collection requested (periodic timer)
[ 44.68 ms] Collector    GC-STOP waiting for 1 active mutator(s) to reach a safepoint
[ 47.50 ms] Mutator-3    THREAD paused at safepoint -- collector holds the world
[ 47.53 ms] Collector    GC-STOP world stopped -- heap is quiescent
[ 47.56 ms] Collector    GC-MARK cycle 1: mark from 5 roots: #1, #2, #3, #4, #8
[ 47.57 ms] Collector    GC-MARK cycle 1: 6 of 8 object(s) reachable
[ 47.58 ms] Collector    RECLAIM cycle 1: swept obj#5 (16B, allocated by Mutator-2)
[ 47.58 ms] Collector    RECLAIM cycle 1: swept obj#7 (40B, allocated by Mutator-2)
[ 47.59 ms] Collector    GC-DONE cycle 1: freed 2 obj / 56B | [#####.....] 136/256B (53.1%)
[ 47.60 ms] Collector    GC-STOP world resumed -- mutators released
[ 47.62 ms] Mutator-3    LINK obj#4 now references obj#8
[ 47.62 ms] Mutator-3    LINK obj#8 now references obj#4
[ 47.62 ms] Mutator-3    THREAD built reference cycle obj#4 <=> obj#8, abandoning both
[ 47.63 ms] Mutator-3    DROP Mutator-3 released obj#4 -- candidate for sweeping
[ 47.63 ms] Mutator-3    DROP Mutator-3 released obj#8 -- candidate for sweeping
[ 53.90 ms] Mutator-1    ALLOC reserved 16B for obj#9 -> pool at 152/256B
[ 53.91 ms] Mutator-1    CREATE obj#9 constructed and rooted in Mutator-1's frame
[ 59.24 ms] Mutator-2    ALLOC reserved 24B for obj#10 -> pool at 176/256B
[ 59.26 ms] Mutator-2    CREATE obj#10 constructed and rooted in Mutator-2's frame
[ 64.68 ms] Mutator-2    DROP Mutator-2 released obj#10 -- candidate for sweeping
[ 65.36 ms] Mutator-3    ALLOC reserved 40B for obj#11 -> pool at 216/256B
[ 65.37 ms] Mutator-3    CREATE obj#11 constructed and rooted in Mutator-3's frame
[ 66.05 ms] Mutator-1    ALLOC reserved 40B for obj#12 -> pool at 256/256B
[ 66.05 ms] Mutator-1    CREATE obj#12 constructed and rooted in Mutator-1's frame
[ 69.72 ms] Mutator-3    DROP Mutator-3 released obj#3 -- candidate for sweeping
[ 71.33 ms] Mutator-1    DROP Mutator-1 released obj#1 -- candidate for sweeping
[ 77.90 ms] Mutator-3    OOM request for 40B refused -- only 0B free
[ 77.92 ms] Mutator-3    GC-STOP collection requested (allocation failure -- reclaiming space)
[ 77.95 ms] Mutator-3    GC-STOP world stopped -- heap is quiescent
[ 77.98 ms] Mutator-3    GC-MARK cycle 2: mark from 4 roots: #2, #9, #11, #12
[ 77.99 ms] Mutator-3    GC-MARK cycle 2: 4 of 10 object(s) reachable
[ 78.00 ms] Mutator-3    RECLAIM cycle 2: swept obj#1 (16B, allocated by Mutator-1)
[ 78.00 ms] Mutator-3    RECLAIM cycle 2: swept obj#3 (16B, allocated by Mutator-3)
[ 78.00 ms] Mutator-3    RECLAIM cycle 2: swept obj#4 (32B, allocated by Mutator-3)
[ 78.00 ms] Mutator-3    RECLAIM cycle 2: swept obj#6 (32B, allocated by Mutator-1)
[ 78.00 ms] Mutator-3    RECLAIM cycle 2: swept obj#8 (16B, allocated by Mutator-3)
[ 78.01 ms] Mutator-3    RECLAIM cycle 2: swept obj#10 (24B, allocated by Mutator-2)
[ 78.02 ms] Mutator-3    GC-DONE cycle 2: freed 6 obj / 136B | [#####.....] 120/256B (46.9%)
[ 78.02 ms] Mutator-3    GC-STOP world resumed -- mutators released

```

Figure 2. Collection under contention. The collector requests a stop-the-world at 44.65 ms, waits for one active mutator, and Mutator-3 parks at a safepoint. Cycle 1 marks six of eight objects and reclaims the abandoned reference cycle. At 66.05 ms the pool reaches 256/256 bytes; at 77.90 ms a 40-byte request is refused, triggering an on-demand collection that frees 136 bytes.

```

python3 mark_sweep_simulation.py  - 3 of 3

[ 78.08 ms] Mutator-3  ALLOC   reserved 40B for obj#13 -> pool at 160/256B
[ 78.11 ms] Mutator-3  CREATE  obj#13 constructed and rooted in Mutator-3's frame
[ 81.88 ms] Mutator-2  ALLOC   reserved 32B for obj#14 -> pool at 192/256B
[ 81.90 ms] Mutator-2  CREATE  obj#14 constructed and rooted in Mutator-2's frame
[ 85.30 ms] Mutator-1  ALLOC   reserved 16B for obj#15 -> pool at 208/256B
[ 85.31 ms] Mutator-1  CREATE  obj#15 constructed and rooted in Mutator-1's frame
[ 88.38 ms] Mutator-1  LINK    obj#12 now references obj#15
[ 88.40 ms] Mutator-1  DROP    Mutator-1 released obj#15 -- candidate for sweeping
[ 89.10 ms] Mutator-2  DROP    Mutator-2 released obj#2 -- candidate for sweeping
[ 89.55 ms] Mutator-3  DROP    Mutator-3 frame unwound, releasing obj#11, obj#13
[ 89.55 ms] Mutator-3  THREAD  finished -- all frames unwound
[ 92.01 ms] Collector  GC-STOP collection requested (periodic timer)
[ 92.02 ms] Collector  GC-STOP world stopped -- heap is quiescent
[ 92.05 ms] Collector  GC-MARK cycle 3: mark from 3 roots: #9, #12, #14
[ 92.06 ms] Collector  GC-MARK cycle 3: 4 of 7 object(s) reachable
[ 92.07 ms] Collector  RECLAIM cycle 3: swept obj#2 (24B, allocated by Mutator-2)
[ 92.07 ms] Collector  RECLAIM cycle 3: swept obj#11 (40B, allocated by Mutator-3)
[ 92.07 ms] Collector  RECLAIM cycle 3: swept obj#13 (40B, allocated by Mutator-3)
[ 92.08 ms] Collector  GC-DONE cycle 3: freed 3 obj / 104B | [#####.....] 104/256B (40.6%)
[ 92.09 ms] Collector  GC-STOP world resumed -- mutators released
[ 96.62 ms] Mutator-2  DROP    Mutator-2 frame unwound, releasing obj#14
[ 96.64 ms] Mutator-2  THREAD  finished -- all frames unwound
[ 97.29 ms] Mutator-1  DROP    Mutator-1 frame unwound, releasing obj#9, obj#12
[ 97.52 ms] Mutator-1  THREAD  finished -- all frames unwound
[ 97.56 ms] MainThread  THREAD  main joined all mutators -- roots are now empty
[ 97.59 ms] Collector  THREAD  collector offline
[ 97.62 ms] MainThread  GC-STOP collection requested (runtime shutdown -- final sweep)
[ 97.62 ms] MainThread  GC-STOP world stopped -- heap is quiescent
[ 97.63 ms] MainThread  GC-MARK cycle 4: mark from 0 roots: none
[ 97.63 ms] MainThread  GC-MARK cycle 4: 0 of 4 object(s) reachable
[ 97.64 ms] MainThread  RECLAIM cycle 4: swept obj#9 (16B, allocated by Mutator-1)
[ 97.64 ms] MainThread  RECLAIM cycle 4: swept obj#12 (40B, allocated by Mutator-1)
[ 97.64 ms] MainThread  RECLAIM cycle 4: swept obj#14 (32B, allocated by Mutator-2)
[ 97.64 ms] MainThread  RECLAIM cycle 4: swept obj#15 (16B, allocated by Mutator-1)
[ 97.65 ms] MainThread  GC-DONE cycle 4: freed 4 obj / 104B | [.....] 0/256B ( 0.0%)
[ 97.65 ms] MainThread  GC-STOP world resumed -- mutators released

=====
Simulation Summary
=====
Pool capacity ..... 256 B
Objects allocated ..... 15
Objects reclaimed ..... 15
Bytes reclaimed ..... 400 B
Peak utilization ..... 256 B (100.0% of pool)
Collection cycles ..... 4
Allocation failures (OOM) .... 1
Live objects at shutdown ..... 0 (pool fully drained)
Bytes still in use ..... 0 B

```

Figure 3. Recovery and shutdown. The retried allocation succeeds immediately after the on-demand collection. Cycle 3 runs on the periodic timer, and the shutdown sweep at 97.62 ms

traces from an empty root set and drains the pool to zero. All fifteen objects allocated were reclaimed.

Runtime Events

The simulation reproduces, in miniature, most of the problems a real concurrent runtime has to solve, and it makes the cost of each solution visible.

6.1 Shared mutable state and race conditions

Every field of the memory pool, the object table, the used-byte counter, the per-thread root sets, and the statistics is reachable from four threads at once. The check-then-act in the allocator is the sharpest example: without mutual exclusion, two threads could both be told a request fits and the pool would silently overflow the budget it exists to enforce. The solution is unglamorous and effective: one lock, held across the whole compound operation rather than around each individual field access. The broader lesson is that the unit of atomicity must be the invariant, not the variable.

6.2 Interference between the collector and the application

A collector that traces a heap while threads mutate it can reach a wrong answer in the most dangerous direction: marking an in-use object as garbage and freeing it. Coordinating this needed a second primitive at a different granularity than the pool lock. The gate permits unlimited concurrency among mutators, since the pool lock already protects the data structures from them, while giving the collector exclusive access when it needs the graph to hold still. Separating phase control from data protection is what keeps the design comprehensible; a single lock covering both would have serialized the mutators against each other for no reason.

6.3 Cyclic garbage

]Reference counting is simpler and reclaims memory promptly, but it cannot free a cycle, because every object in one retains a non-zero count. The two cycles built deliberately in this run — 56 bytes and 48 bytes — would have leaked under counting and were reclaimed under tracing. On a device with a 256-byte budget, leaking 104 bytes across a few operations would be fatal within seconds. This is the concrete reason the prototype traces rather than counts.

6.4 Exhaustion as a normal condition, not a crash

Under concurrency the pool can fill between one thread's decision to allocate and another's. Treating that as a fatal error would make the runtime unusable. Instead a refused allocation returns a sentinel, the thread requests a collection and retries once, and the trace shows this working: the request for 40 bytes at 77.90 ms is refused with zero bytes free, the resulting collection frees 136 bytes, and the retry succeeds within 200 microseconds. Allocation failure becomes a scheduling signal rather than a failure mode.

6.5 Collector starvation and deadlock

Two coordination bugs were designed out rather than debugged out. Making the gate writer-preferring prevents a continuous stream of allocations from postponing collection forever. Fixing a global lock order, and ensuring a thread never requests a collection while it is itself counted as an active mutator, prevents both the two-lock deadlock and the subtler case of a thread waiting on its own participation. Both hazards are invisible in a single-threaded design and neither produces a useful error message when it occurs — the program simply stops.

6.6 Nondeterminism and how to test it

Because thread scheduling is not reproducible, the program does not produce identical output twice even with object sizes seeded. This is the practical difficulty that makes concurrency bugs expensive: a defect may appear in one run out of hundreds. The response is to assert invariants rather than exact output. Running the simulation fifteen times, every run allocated fifteen objects and reclaimed all fifteen, and the pool drained to zero bytes at shutdown. What varied was the timing of collection, not its correctness: runs ended with four or five collection cycles, one or two allocation failures, and peak utilization between 232 and 256 bytes. Correctness held constant while performance characteristics moved, which is exactly the property a memory manager should have.

6.7 The pause-time tradeoff

Stop-the-world is correct and simple, and the trace shows what it costs: at 44.68 ms the collector blocked all application progress for roughly three milliseconds waiting for a single mutator to reach a safepoint, before doing any useful work. On a three-thread toy that is negligible. Scaled to a real heap with many threads and hundreds of thousands of objects, mark time grows with the live set and pause times become the dominant complaint against tracing collectors. This is the pressure that produced generational collection, which exploits the observation that most objects die young; incremental and concurrent collectors, which interleave marking with mutation using write barriers; and region-based schemes such as Rust's ownership model, which avoid runtime tracing altogether. A production version of this embedded runtime would most plausibly move to an incremental collector to bound worst-case pause length.

Limitations

- The pool tracks a single scalar of used bytes, so it models capacity but not fragmentation; a real allocator can fail a request while nominally holding enough free memory, and no compaction is simulated.
- Allocations are simulated accounting rather than real memory, so the program demonstrates the algorithm rather than measuring true allocator performance.
- Because of the global interpreter lock, the threads interleave rather than execute in parallel on separate cores. The synchronization hazards are identical, but the timing profile of a genuinely parallel runtime would differ.

Conclusion

The simulation meets its goal: objects are created and reclaimed across concurrent threads, shared state is protected by locks chosen at appropriate granularities, and a mark-and-sweep cycle correctly reclaims unreachable memory including reference cycles that a counting collector could not free. More usefully, it makes the tradeoffs legible. Safety under concurrency was obtained by constraining when threads may run, and that constraint is visible in the trace as measurable pause time. That exchange correctness purchased with throughput is the central design tension in every real memory manager, and it is the reason garbage collection remains an active area of language runtime research rather than a solved problem.

Acknowledgment of AI Use

To complete this activity, I utilized a generative AI tool—Claude Code (powered by Anthropic's Claude Opus 5). I used the AI primarily for architectural assistance, specifically to help design the simulation's structure and decompose the system into its core components: HeapObject, MemoryPool, and MutatorGate.